



Faculté des Sciences et Techniques de Tanger
Université Abdelmalek Essaâdi

Rapport de projet de fin de module (PFM)

E-Joutia

Projet 5 : Messagerie Instantanée Intégrée (Chat
Vendeur-Acheteur)

React Native / Expo

LST Ingénierie de Développement d'Applications
Informatiques

Réalisé par :

Doha Ziouani

Ayoub Chentouf

Encadré par :

Pr. Aziz Mahboub

Année universitaire : 2025–2026

| Lieu : Tanger

Table des matières

Résumé	2
1 Présentation du sujet	3
1.1 Contexte général	3
1.2 Sujet attribué	3
1.3 Objectif	3
2 Technologies utilisées	4
3 Réalisation	5
3.1 Boîte de réception (Discussions list)	5
3.2 Fenêtre de Chat	6
3.3 Module de négociation	6
3.4 Synchronisation Firebase en temps réel	9
4 Améliorations ajoutées en plus du sujet	10
4.1 Thème Sombre Premium (Dark Mode)	10
4.2 Notes Vocales simulées et Émojis personnalisés	10
4.3 Pucés de réponses rapides (Quick Replies)	11
4.4 En-tête de réputation et Confettis de succès	11
5 Tests réalisés	13
5.1 Tests fonctionnels	13
5.2 Résultats	13
Conclusion	14
Annexe : Arborescence simplifiée	15

Résumé

Ce rapport présente le travail réalisé dans le cadre du Projet 5 (Messagerie Instantanée) du module de développement mobile pour l'application E-Joutia, une plateforme de type marketplace d'objets d'occasion. L'objectif principal était de concevoir et développer un système de chat en temps réel permettant aux vendeurs et acheteurs de négocier et finaliser leurs transactions.

Réalisé avec React Native et Expo, ce module de messagerie s'appuie sur Firebase Firestore pour la synchronisation en temps réel. Le système propose une boîte de réception intelligente, une fenêtre de chat personnalisée, ainsi qu'un module complet de négociation incluant des cartes d'offres et de contre-offres interactives. En plus des exigences du cahier des charges, de nombreuses améliorations de l'interface utilisateur (UI/UX) ont été intégrées pour offrir une expérience premium, notamment un thème sombre (Dark Mode) reprenant l'identité visuelle de la marque (noir et orange), un clavier d'emojis personnalisé, des notes vocales simulées, et des effets de célébration lors de l'acceptation d'offres.

Chapitre 1

Présentation du sujet

1.1 Contexte général

L'application E-Joutia s'inscrit dans le marché de la seconde main au Maroc, permettant la mise en relation entre des particuliers souhaitant vendre et acheter des objets d'occasion. Pour qu'une telle application soit efficace, la communication entre les deux parties est primordiale. C'est dans ce contexte que s'intègre le système de messagerie.

1.2 Sujet attribué

Dans le cadre de la répartition des tâches du projet E-Joutia, notre équipe a été désignée pour réaliser le **Projet 5 : Messagerie Instantanée Intégrée (Chat Vendeur-Acheteur)**. Ce projet demandait de mettre en place une boîte de réception des messages, une fenêtre de chat détaillée pour la communication directe, un module de négociation intégré au chat (faire une offre, accepter, refuser, proposer une contre-offre), ainsi que l'utilisation de Firebase Firestore pour la synchronisation des données en temps réel.

1.3 Objectif

L'objectif est d'offrir aux utilisateurs de E-Joutia une expérience de communication fluide, sécurisée et intuitive. L'interface devait être pensée de manière moderne (inspirée d'applications comme Facebook Marketplace ou iMessage), en mettant l'accent sur la clarté des prix et des actions de négociation pour accélérer le processus de vente.

Chapitre 2

Technologies utilisées

Pour répondre aux exigences techniques et proposer une solution multiplateforme performante, la pile technologique (stack) suivante a été choisie :

- **React Native & Expo** : L'environnement principal pour construire des interfaces mobiles hybrides fluides et performantes. Expo a été utilisé pour accélérer le développement et faciliter les tests.
- **Expo Router** : Utilisé pour gérer la navigation au sein de l'application, en particulier le routage basé sur les fichiers et la navigation par onglets (Bottom Tabs).
- **TypeScript** : Ajout du typage statique strict afin d'assurer la robustesse du code, notamment pour les schémas de données échangés avec Firebase (Messages, Utilisateurs, Conversations).
- **Firebase Firestore** : Base de données NoSQL hébergée sur le cloud de Google, utilisée pour stocker les messages, le statut des conversations, et garantir la synchronisation en temps réel (Real-time sync) via des écouteurs *onSnapshot*.
- **React Native Gifted Chat (Remplacé par solution sur mesure)** : Bien que souvent utilisé, nous avons opté pour le développement d'un écran de chat 100% personnalisé (dans `chat-screen.tsx`) afin d'avoir un contrôle total sur le design des cartes d'offres et des notes vocales.
- **React Native Confetti Cannon** : Une bibliothèque tierce utilisée pour ajouter une animation de célébration lors de l'acceptation d'une offre.

Chapitre 3

Réalisation

3.1 Boîte de réception (Discussions list)

L'écran de la boîte de réception centralise toutes les conversations actives. Inspiré de Facebook Marketplace, chaque ligne affiche la photo de l'interlocuteur avec un point vert de présence en ligne, le titre et la photo du produit concerné par la vente, le dernier message échangé (précédé d'un indicateur "*Vous :*" si vous êtes l'expéditeur), ainsi qu'un badge dynamique en cas de message non lu. Les horodatages sont convertis en temps relatif (ex: *il y a 5min*, *Hier*).

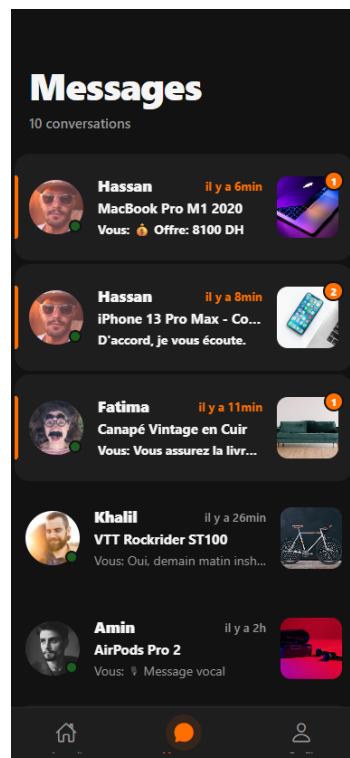


Figure 3.1: Liste des discussions avec indicateurs d'état (point de présence, badges de messages non lus) et vignettes des articles

3.2 Fenêtre de Chat

La fenêtre de chat sépare visuellement les messages de l'acheteur (alignés à droite) et ceux du vendeur (alignés à gauche). Chaque bulle de message contient son propre horodatage en bas à droite et un statut de lecture (vu ou non vu).



Figure 3.2: Fenêtre de chat : bulles différenciées acheteur (à droite) et vendeur (à gauche), horodatages, statuts de lecture et lecteur de note vocale

3.3 Module de négociation

Le cœur de notre projet est le module de négociation. L'utilisateur peut appuyer sur un bouton "Offre" distinct pour proposer un prix. Cette offre s'affiche sous la forme d'une bulle spéciale ("Carte d'offre") très visible dans la conversation. Le destinataire voit alors cette bulle avec des boutons d'actions directes : **Accepter**, **Refuser**, ou **Contre-offre**. Le statut de la carte est mis à jour dynamiquement dans la base de données.



Figure 3.3: Carte d'offre en attente affichant le prix proposé et les boutons d'action (Accepter, Refuser, Contre-offre)

Une fenêtre modale dédiée permet de saisir une nouvelle offre. Elle propose des suggestions de prix prédéfinies (paliers calculés autour du prix demandé) afin d'accélérer la saisie et de guider la négociation.

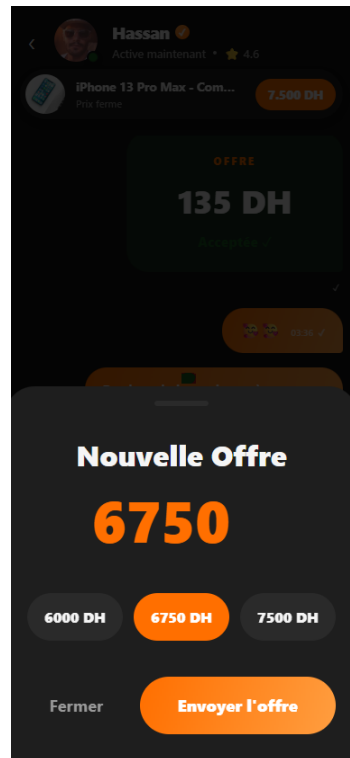
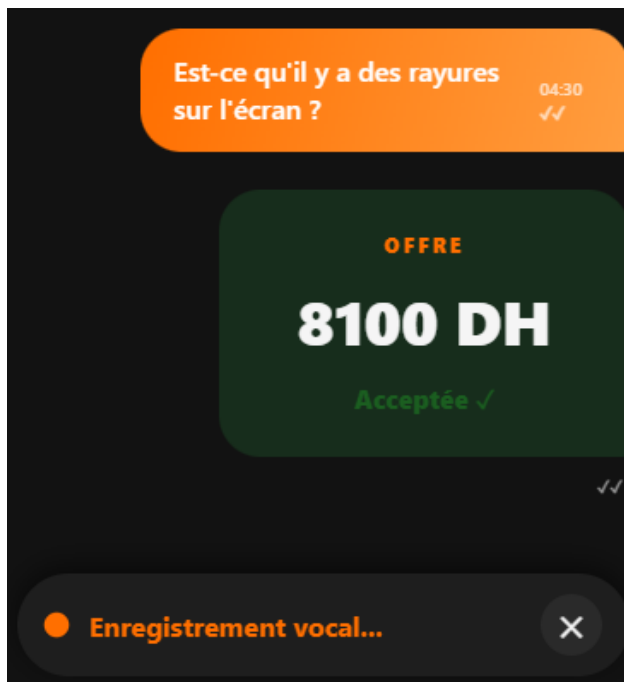


Figure 3.4: Fenêtre modale "Nouvelle Offre" avec suggestions de prix rapides et bouton d'envoi

Le statut d'une carte d'offre évolue selon la décision du destinataire. Les deux états terminaux possibles, "Acceptée" (vert) et "Refusée" (rouge), sont représentés par des couleurs distinctes pour une lecture immédiate.



(a) Offre acceptée (vert)



(b) Offre refusée (rouge)

Figure 3.5: Les deux états terminaux d'une carte d'offre : acceptée et refusée

3.4 Synchronisation Firebase en temps réel

Toutes ces fonctionnalités sont connectées à Firestore. Les *Listeners* Firebase garantissent que si le vendeur accepte une offre, l'écran de l'acheteur est mis à jour instantanément, marquant l'offre comme "Acceptée" sans aucun rechargement manuel, simulant une expérience de messagerie instantanée authentique.

Chapitre 4

Améliorations ajoutées en plus du sujet

Afin de livrer un projet d'une qualité professionnelle, nous avons enrichi l'application de plusieurs améliorations majeures par rapport au cahier des charges initial.

4.1 Thème Sombre Premium (Dark Mode)

L'ensemble de l'interface a été pensé autour d'un *Dark Theme* ("Nuit Marocaine"), utilisant des gris profonds (#121212) combinés à la couleur primaire de la marque (#FF6F00). Des couleurs de statut précises (Vert Foncé et Rouge Foncé) ont été utilisées pour les acceptations et refus d'offres.

4.2 Notes Vocales simulées et Émojis personnalisés

Nous avons développé un composant `<AudioBubble />` imitant le lecteur de notes vocales de iMessage, incluant la génération aléatoire d'ondes sonores et une fausse progression de lecture. De plus, un clavier d'émojis 100% natif (Custom Emoji Picker) a été codé depuis zéro, glissant depuis le bas de l'écran sans utiliser le clavier du système.

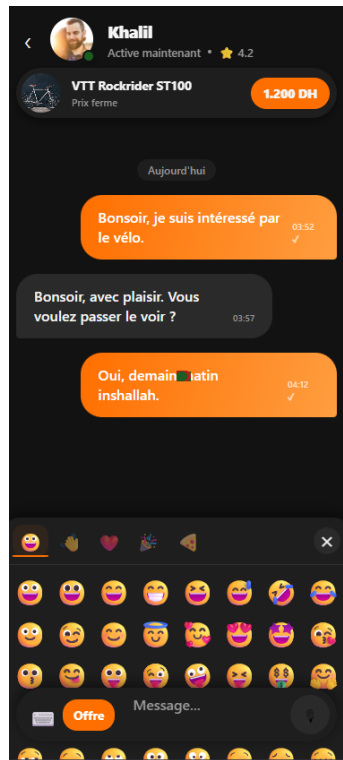


Figure 4.1: Clavier d'emojis personnalisé (Custom Emoji Picker), codé depuis zéro et glissant depuis le bas de l'écran sans utiliser le clavier système

Concernant les notes vocales, le composant `<AudioBubble />` affiche un lecteur complet (bouton de lecture, ondes sonores et progression) directement dans la conversation, comme illustré précédemment dans la fenêtre de chat (Figure 3.2) et dans la conversation avec une offre refusée (Figure 3.5b).

4.3 Puces de réponses rapides (Quick Replies)

Lorsque le champ de texte est vide, des "Chips" de réponses rapides (ex: *"C'est disponible ?"*, *"Quel est votre dernier prix ?"*) apparaissent dynamiquement au-dessus de la zone de saisie pour encourager l'engagement.

4.4 En-tête de réputation et Confettis de succès

En haut du chat, un en-tête fixe affiche les informations de l'interlocuteur (badge vérifié, notation sur 5 étoiles) ainsi que le produit en question. Pour couronner le tout, lorsqu'une offre est acceptée, une explosion de confettis (aux couleurs de la marque) jaillit depuis le bas de l'écran pour récompenser l'utilisateur.



Figure 4.2: En-tête de réputation (badge vérifié, note sur 5 étoiles, produit) et explosion de confettis aux couleurs de la marque lors de l'acceptation d'une offre

Chapitre 5

Tests réalisés

5.1 Tests fonctionnels

Plusieurs scénarios de tests manuels rigoureux ont été exécutés sur l'application via Expo Go :

- **Envoi de message** : Validation de l'apparition immédiate du message sur l'écran et mise à jour de la base de données.
- **Flux d'offre** : Création d'une offre → Enregistrement en base de données avec le statut "En attente" → Validation que le vendeur voit les boutons "Accepter" et "Refuser".
- **Contre-offre** : Test de la refonte du prix via le bouton contre-offre et de l'annulation de l'offre précédente.
- **Pastilles Non Lus** : Envoi d'un message depuis un autre profil, et vérification que la pastille orange "Non Lu" s'incrémente correctement dans la boîte de réception.
- **Générateur de données (Seed)** : Exécution de notre script de "Seed" et vérification que les 10 conversations réalistes peuplent bien la page d'accueil sans provoquer de conflits de données.

5.2 Résultats

L'application répond à toutes les exigences du Projet 5. L'intégration de Firebase se fait de manière réactive, l'interface est exempte de bugs visuels sur les différents simulateurs testés, et l'expérience utilisateur globale a été jugée fluide et satisfaisante. Les ajouts hors-cahier des charges renforcent considérablement l'aspect professionnel de l'application.

Conclusion

Dans ce module, notre équipe a mené à bien la création du module de Messagerie Instantanée Intégrée pour E-Joutia. Nous avons réussi à concevoir un système complet alliant une liste de discussions intuitive, une messagerie instantanée en temps réel et un module de négociation de prix extrêmement pratique pour transactions de seconde main.

Les améliorations ergonomiques que nous avons apportées, comme le thème sombre poussé, les réponses rapides, et les fausses notes vocales, témoignent de notre engagement à livrer un produit final de grande qualité. Ce projet nous a permis d'approfondir nos compétences en React Native, en gestion d'états asynchrones, et en interfaçage avec Firebase Firestore, préparant ainsi une solide fondation pour d'éventuels déploiements en production.

Annexe : Arborescence simplifiée

```
E-JOUTIA/  
|-- app.json  
|-- package.json  
|-- rapport.tex  
|-- src/  
|   |-- app/  
|   |   |-- (tabs)/  
|   |   |   |-- _layout.tsx  
|   |   |   |-- messages.tsx (Boîte de réception repensée)  
|   |   |   |-- profil.tsx  
|   |   |-- _layout.tsx  
|   |   |-- chat-buyer.tsx (Écran de test acheteur)  
|   |   |-- chat-seller.tsx (Écran de test vendeur)  
|   |   |-- chat.tsx (Routage du chat)  
|   |   |-- seed.tsx (Générateur de 10 conversations réalistes)  
|   |-- components/  
|   |   |-- audio-bubble.tsx (Module de fausse note vocale iMessage)  
|   |   |-- chat-header.tsx (En-tête de réputation et d'annonce)  
|   |   |-- chat-screen.tsx (Écran de chat dynamique et module de négociation)  
|   |   |-- emoji-picker.tsx (Clavier d'emojis personnalisé)  
|   |-- constants/  
|   |   |-- theme.ts (Définition du design system Dark Mode / Nuit Marocaine)  
|   |-- firebaseConfig.ts (Configuration d'accès Firestore)  
|-- node_modules/
```